

1. SYNTHÈSE

Objectif SAE : Développer une application complète de gestion de vols et d'aéroports (3 couches : API Flask, Web SPA, Mobile Flutter). Je me suis principalement occupé de la partie BD ainsi que de l'API Flask avec la gestion d'Oracle.

2. ANALYSE DE CODE

```
27 @app.route('/api/vols', methods=['POST'])
28 def add_vol():
29     nouveau_vol = request.get_json()
30
31     if not nouveau_vol or "compagnie" not in nouveau_vol:
32         return jsonify({"erreur": "Données invalides"}), 400
33
34     try:
35         conn = db.get_connection()
36         cursor = conn.cursor()
37
38         # SÉCURITÉ : Vérifier si le vol existe déjà
39         cursor.execute("SELECT COUNT(*) FROM Vol WHERE compagnie = :1 AND num_vol = :2",
40                        (nouveau_vol['compagnie'], nouveau_vol['num_vol']))
41         if cursor.fetchone()[0] > 0:
42             return jsonify({"erreur": "Ce vol (Même compagnie et même numéro) existe déjà !"}), 400
43
44         cursor.execute("""
45             INSERT INTO Vol (compagnie, num_vol, date_heure_dep, id_aero_dep, id_aero_arr)
46             VALUES (:1, :2, SYSTIMESTAMP, :3, :4)
47             """, (nouveau_vol['compagnie'], nouveau_vol['num_vol'], nouveau_vol['depart'], nouveau_vol['arrivee']))
48
49         conn.commit()
50         cursor.close()
51         conn.close()
52
53         return jsonify({"message": "Vol ajouté avec succès", "vol": nouveau_vol}), 201
```

Difficultés identifiées :

Gestion connexions Oracle : ouvrir/fermer

Validation doublons : balance entre UX (frontend) et sécurité (API)

Codes HTTP REST : déterminer 200/201/400/500 selon le cas

Solutions appliquées :

Connexion explicite + fermeture systématique à chaque requête

Double validation (frontend UX + backend sécurité)

Codes HTTP conformes au standard REST

3. DÉMONSTRATION DE COMPÉTENCES

RÉALISER

AC21.01 : Spécifications fonctionnelles/non-fonctionnelles (DOSSIER_SPECIFICATION.md)

AC21.02 : Accessibilité/ergonomie (interfaces intuitives, navigation claire)

AC21.03 : Bonnes pratiques (séparation, nommage, DRY, validation)

AC21.04 : Tests (pytest 5/5, validation testée, couverture doc.)

OPTIMISER

AC22.02 : Algo complexe (CTE récursive pour graphe de vols)

AC22.04 : Impact environnemental (virtualisation, légèreté)

ADMINISTRER

AC23.01 : Apps communicantes (API REST + 2 clients Web/Mobile)

AC23.02 : Serveurs virtualisés (venv, Flask, http.server, Oracle)

GÉRER

AC24.02 : Sécurité (validation, SQL param., CORS)

AC24.03 : Restitution données (JSON, Web, Flutter visual)

AC24.04 : Données hétérogènes (texte, nombre, timestamp, joins)

COLLABORER

AC26.02 : Intégration équipe (code lisible, conventions, structure std)

AC26.04 : Rendre compte (ce rapport, docs accessibles, test doc.)